

HDOC - Day 28

Variable-Length 2D Arrays and Row-Major Processing

Algorithm

1. Program 1: Read rows and columns repeatedly until both are positive.
2. Declare a variable-length 2D integer array using the accepted dimensions.
3. Read elements column by column using columns as the outer loop and rows as the inner loop.
4. Print the same 2D array in row-major order using one loop and a linear index.
5. Program 2: Read dimensions and elements into another variable-length 2D array.
6. Copy each element into a 1D array in row-major order and display both arrays.
7. Stop.

Test Case Table

Test	Program	Input	Expected output
1	1	r=2,c=3; column input: 1 4 2 5 3 6	Row-major: 1 2 3 4 5 6
2	1	invalid r=0,c=3 then r=2,c=2	Input repeated; valid matrix printed row-major
3	2	2x3: 10 20 30 / 40 50 60	1D: 10 20 30 40 50 60
4	2	3x2: 1 2 / 3 4 / 5 6	1D: 1 2 3 4 5 6

C Program

```
/* Program 1 */
#include <stdio.h>
int main(){
    int r,c,i,j;
    do{printf("Enter rows and columns: ");scanf("%d%d",&r,&c);}while(r<=0||c<=0);
    int a[r][c];
    printf("Enter elements column by column:\n");
    for(j=0;j<c;j++) for(i=0;i<r;i++) scanf("%d",&a[i][j]);
    printf("Row-major order: ");
    for(i=0;i<r*c;i++) printf("%d ",a[i/c][i%c]);
    return 0;
}

/* Program 2 */
#include <stdio.h>
int main(){
    int r,c,i,j,k=0; printf("Enter rows and columns: ");scanf("%d%d",&r,&c);
    int a[r][c],b[r*c];
    printf("Enter 2D array elements:\n");
    for(i=0;i<r;i++) for(j=0;j<c;j++) scanf("%d",&a[i][j]);
    for(i=0;i<r;i++) for(j=0;j<c;j++) b[k++]=a[i][j];
    printf("2D array:\n");
    for(i=0;i<r;i++){for(j=0;j<c;j++)printf("%d ",a[i][j]);printf("\n");}
    printf("1D array: "); for(i=0;i<r*c;i++)printf("%d ",b[i]);
    return 0;
}
```

Compilation, Execution & Testing

Test Case 1 - Column-wise input, row-major output

```
puppie@Anvikas-MacBook-Air Desktop % clang day28_1.c -o day28_1
puppie@Anvikas-MacBook-Air Desktop % ./day28_1
Enter rows and columns: 2 3
Enter elements column by column:
1 4 2 5 3 6
Row-major order: 1 2 3 4 5 6
```

Test Case 2 - Invalid dimensions repeated

```
puppie@Anvikas-MacBook-Air Desktop % ./day28_1
Enter rows and columns: 0 3
Enter rows and columns: 2 2
Enter elements column by column:
1 3 2 4
Row-major order: 1 2 3 4
```

Test Case 3 - Copy 2D to 1D

```
puppie@Anvikas-MacBook-Air Desktop % clang day28_2.c -o day28_2
puppie@Anvikas-MacBook-Air Desktop % ./day28_2
Enter rows and columns: 2 3
Enter 2D array elements:
10 20 30 40 50 60
2D array:
10 20 30
40 50 60
1D array: 10 20 30 40 50 60
```

Test Case 4 - Second 2D to 1D case

```
puppie@Anvikas-MacBook-Air Desktop % ./day28_2
Enter rows and columns: 3 2
Enter 2D array elements:
1 2 3 4 5 6
2D array:
1 2
3 4
5 6
1D array: 1 2 3 4 5 6
```

Result: Both C programs using variable-length 2D arrays were implemented and tested successfully for column-wise input, row-major output, and 2D-to-1D copying.